

Laboratorio 7

Determinación de la Temperatura crítica
del modelo de Ising 2D con redes neuronales
de scikit-learn

Notas

Valentina Pérez Cadavid

2026-2

Se entrenará una red neuronal clasificadora con scikit-Learn y a usarla como un termómetro para localizar la temperatura crítica T_c de la transición de fase ferromagnética del modelo de Ising en 2D

1. Física: Transición de fase del modelo de Ising 2D.

El modelo de Ising en una red $2 \times L$ con acoplamiento ferromagnético $J > 0$ tiene energía

$$E(\{s_i\}) = -J \sum_{\langle i,j \rangle} s_i s_j \quad s_i \in \{-1, 1\}$$

donde la suma es sobre pares de primeros vecinos.

$$J = k_B = 1.$$

El sistema infinito presenta una transición de fase de segundo orden a la temperatura crítica de Onsager

$$T_c = \frac{2}{\ln(1+\sqrt{2})} \approx 2.269$$

ordenado | Desordenado
 T_c

→ $T < T_c$: fase ordenada (ferromag) $m \gg 0 \rightarrow y = 0$

→ $T > T_c$: fase desordenada (paramag) $\rightarrow y = 1$
 $m \approx 0$

Se calculará m para saber si la configuración es ordenada o desordenada. Esto se hará con una red neuronal a la cual se le mostrará configuraciones muy frías ($\underbrace{m \approx 0}_0$) y muy calientes ($\underbrace{m \gg 0}_1$)

La red aprenderá a distinguir el "patrón" visual de una fase u otra

Después se le mostrarán configuraciones que están cerca de T_c . Como el patrón va cambiando gradualmente de "se ve ordenado" a "se ve desordenado", la probabilidad que de la red va a variar gradualmente de 0 a 1 al aumentar T . El punto donde esa probabilidad promedio cruza 0.5, es la estimación de T_c .

3. Datos: Configuraciones Lejos de T_c

→ `ising_dataset.npz` : archivo comprimido de numpy que tiene varios arreglos identificados por nombres, generado en el notebook: `Sesion07_bDatos_Ising`, simulando el modelo de Ising con el algoritmo de Metrópolis para generar configuraciones aleatorias de espines.

Campo	Forma	Significado
<code>x</code>	<code>(1600, 16, 16)</code>	Configuraciones de espines $s_i \in \{-1, +1\}$
<code>y</code>	<code>(1600,)</code>	Etiqueta de fase: 0 = ordenada (T baja), 1 = desordenada (T alta)
<code>T</code>	<code>(1600,)</code>	Temperatura de cada configuración
<code>m</code>	<code>(1600,)</code>	Magnetización por sitio ϕ
<code>realizacion</code>	<code>(1600,)</code>	Índice de cadena Monte Carlo independiente (0..3)
<code>Tc, L, frontera</code>	escalares	Metadatos (T_c exacto, tamaño, tipo de frontera)

→ Tiene 1600 configuraciones de una red 16×16

→ m es un $0 \leq \phi \leq 1$

→ de cuál de las 4 cadenas de Monte Carlo salió cada configuración

Tipo de frontera: periódica

$L = 16$

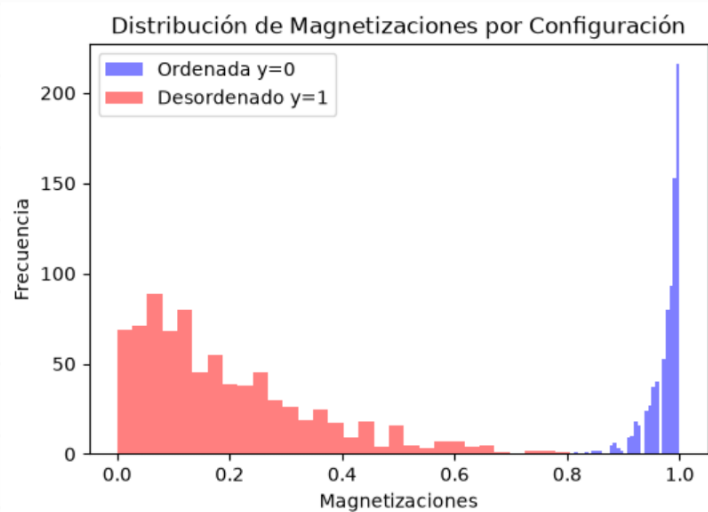
Ejercicio 1: Exploración

1. ¿Qué temperaturas tienen etiqueta $y=0$ y cuál $y=1$?
Dónde está el hueco?

$$y = [0 \ 1 \ 0 \ 1 \ 1 \ 0 \ 1 \ 1 \ 1 \ 0]$$

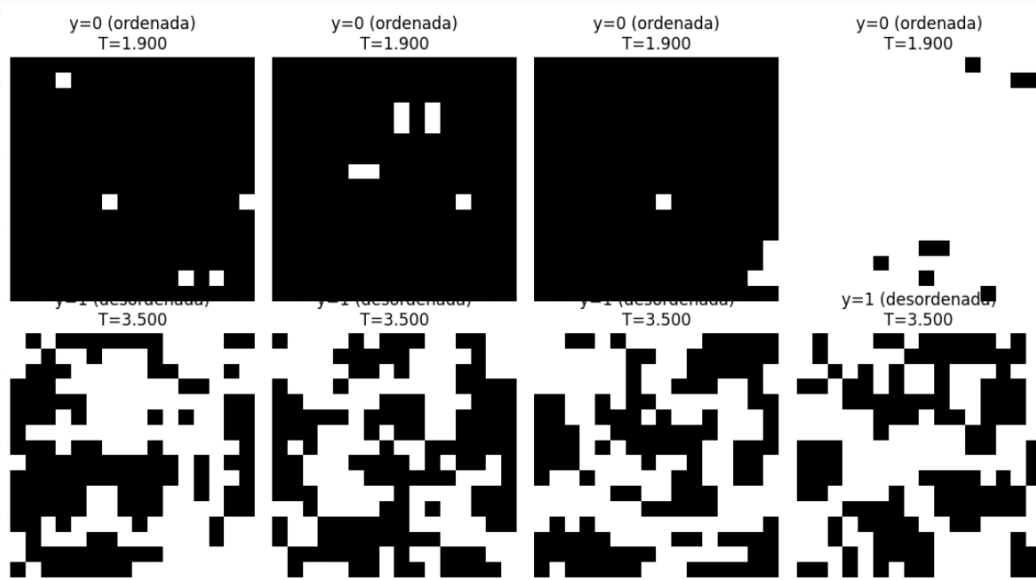
$$T = [T_1 \ T_2 \ T_3 \ T_4 \ T_5 \ T_6 \ T_7 \ T_8 \ T_9 \ T_{10}]$$





Las dos fases se logran separar bien

La fase desordenada forma una distribución bastante ancha, lo que indica la correlación entre espines



4. Partición de los datos : por realización

Para la generación de los datos (notebook Sesión07_bDatos_Ising) se usó:

→ Para una T fija la simulación no genera 1600 configuraciones desde 0. cada vez. En lugar de esto, arranca en una configuración, hace barridos volteando espines al azar y guarda la configuración.

Cada configuración se forma partiendo de la anterior. Así que están correlacionadas (autocorrelación de Monte Carlo)

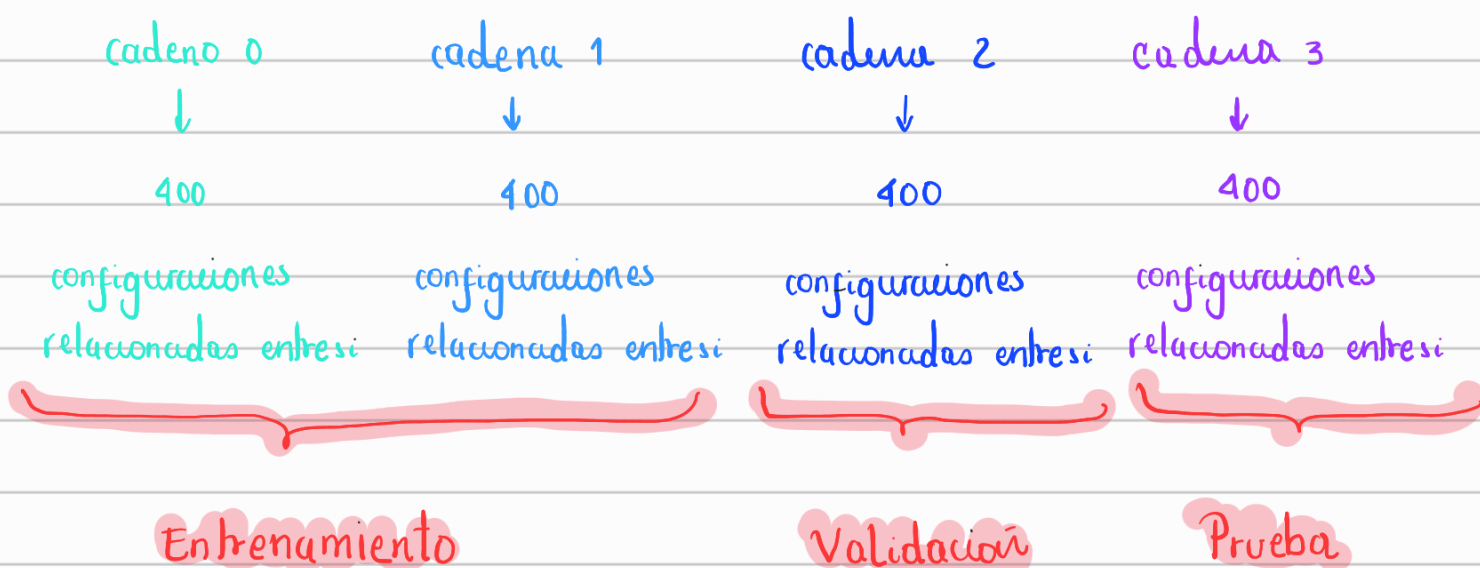
$$N_{TOTAL} = N_{realizaciones} \times N_{Tvalidas} \times N_{conf. por T}$$

$$4 \times 10 \times 40 = 1600$$

- 4 realizaciones: (cadenas 0,1,2,3)
- 10 Temperaturas válidas: 5 frías y 5 calientes
- 40 configuraciones guardadas por cada temperatura

Hay 400 imágenes por cadena:

Cada cadena, es una secuencia de pasos lógicos que involucran aleatoriedades diferentes.



Ejercicio 2 : Construir los conjuntos de datos

- `realizacion = [0 3 1 0 2 1 ...]` (1600,)
- `np.isin(realización, [0,1])` : recibe el arreglo `realización` y una lista de valores a buscar.
 Por cada elemento de `realizacion`, revisa si ese valor está contenido en la lista `[0,1]`.
 El resultado es un nuevo arreglo, del mismo tamaño que `realizacion` con `True` o `False`.
`[ T T F F T F T F F T T ... ]`

→ `def aplanar(A)`: recibe un arreglo A (una pila de imágenes 16×16) y lo convierte en una pila de vectores planos de 256

`A.shape[0]`: toma el primer número de la forma de A (800, 16, 16)

`mask_train` = [T F F T T T F ...] (1 x 1600)

`mask_val` = [T T F T F T F ...] (1 x 1600)

`mask_test` = [F F T F T F T ...] (1 x 1600)

→ `X[mask_train]`: Crea una lista que contiene los valores en donde había un True. Por tanto, es una lista mas pequeña.

Es decir, crea una lista con las configuraciones obtenidas de las realizaciones 0 y 1

[   
    ...] } se debe aplanar
(800, 16, 16)

→ `aplanar(X[mask_train])`: devuelve un arreglo (800, 256)

→ `X[mask_val]`: Crea lista que contiene solo las config. generadas con la realización 2.

[   
    ...] } se debe aplanar
(400, 16, 16)

→ `aplanar(X[mask_val])`: devuelve un arreglo (400, 256)

→ `x[mask_test]`: crea lista que contiene solo las config generadas con la realización 3

[] } se debe aplanar

(400, 16, 16)

→ `aplanar(x[mask_test])`: devuelve un arreglo (400, 256)

Para `y`: $y = [0 \ 1 \ 0 \ 0 \ 1 \dots]$

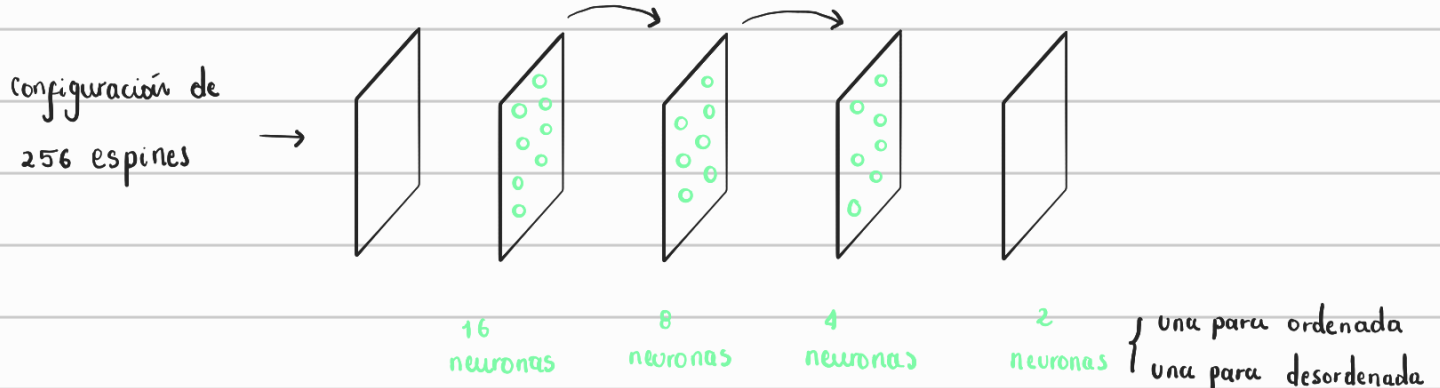
`realizacion` = [1 0 2 2 3 ...]

`mask_val` = [F F T T F ...]

`y[mask_val]` = [0 0 ...]

T sigue el mismo razonamiento.

5. Entrenar el MLP Classifier



→ Cada **neurona** hace una combinación matemática de los números que recibió. Multiplica cada uno por un peso, los suma y aplica una función no lineal. Y pasa su resultado a la siguiente capa.

Al final sale un número que se interpreta como qué tan seguro está el modelo de que es una fase ordenada / desordenada

Los pesos que multiplican cada conexión entre neuronas son los que el modelo aprende durante el entrenamiento. Al inicio son # al azar y se van ajustando para que la red acierte

5. Entrenar el MLPClassifier

Argumento	Valor	Qué controla
<code>hidden_layer_sizes=(16, 8, 4)</code>	3 capas ocultas	Arquitectura: $256 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2$
<code>activation="relu"</code>	ReLU	No linealidad de las capas ocultas
<code>solver="adam"</code>	Adam	Optimizador basado en gradiente estocástico
<code>learning_rate_init=0.001</code>	0.001	Tamaño de paso inicial
<code>max_iter=1000</code>	1000	Máximo de épocas (puede parar antes por tolerancia)
<code>random_state=42</code>	42	Reproducibilidad (init de pesos y shuffling)

La capa de salida (2 neuronas, *softmax*) y la pérdida (*log-loss*) las añade *scikit-learn* automáticamente al ver que *y* tiene dos clases enteras. En *NN_Cero.ipynb* esto era la *sigmoide* final + *cost_Function*; en *PyTorch* era `nn.Linear(..., 1) + BCEWithLogitsLoss`.

→ Entada que va pasando la información de los números, 2 opciones

→ **MLPClassifier** construye automáticamente todas las conexiones entre capas consecutivas : $256 \rightarrow 16$, $16 \rightarrow 8$, $8 \rightarrow 4$, $4 \rightarrow 2$, y las llena de pesos iniciales al azar.

→ **ReLU** : Rectified Linear Unit es una función matemática que si el # que recibe es positivo , lo deja igual. Si es negativo lo convierte en 0.

Si cada neurona solo hiciera sumas sin esta función, toda la red, sin importar cuantas capas tenga, se reduciría matemáticamente a una simple combinación lineal, incapaz de aprender patrones.

ReLU introduce la no linealidad que le permite a la red aprender relaciones complicadas

→ **solver = "adam"** : Algoritmo que decide como ajustar los pesos en cada paso de entrenamiento.

Es una versión mejorada del descenso de gradiente

→ `learning_rate_init = 0.001` : Cada vez que la red ajusta los pesos, no salta directo a la solución sino que da un paso pequeño en la dirección que reduce el error

→ `max_iter = 1000` : Una época es una pasada completa por todos los datos de entrenamiento ajustando pesos. Le dice al modelo que debe entrenar como máximo 1000 veces sobre los datos

→ `random_state = 42` : Dice cómo se inicializan los pesos aleatoriamente al inicio,

→ la capa de salida tiene 2 neuronas

Ejemplo:

2 neuronas { una para ordenada → produce un # crudo (logit) 1.2
una para desordenada → produce un # crudo (logit) 0.3 } Significan en una escala arbitraria, que tanta evidencia acumuló cada camino de la red a favor de cada clase

Softmax: convierte esos 2 números en probabilidades que suman 1.

1.2 → 71% ordenada
0.3 → 29% desordenada } Es lo que se obtiene al llamar `model.predict_proba(x)`

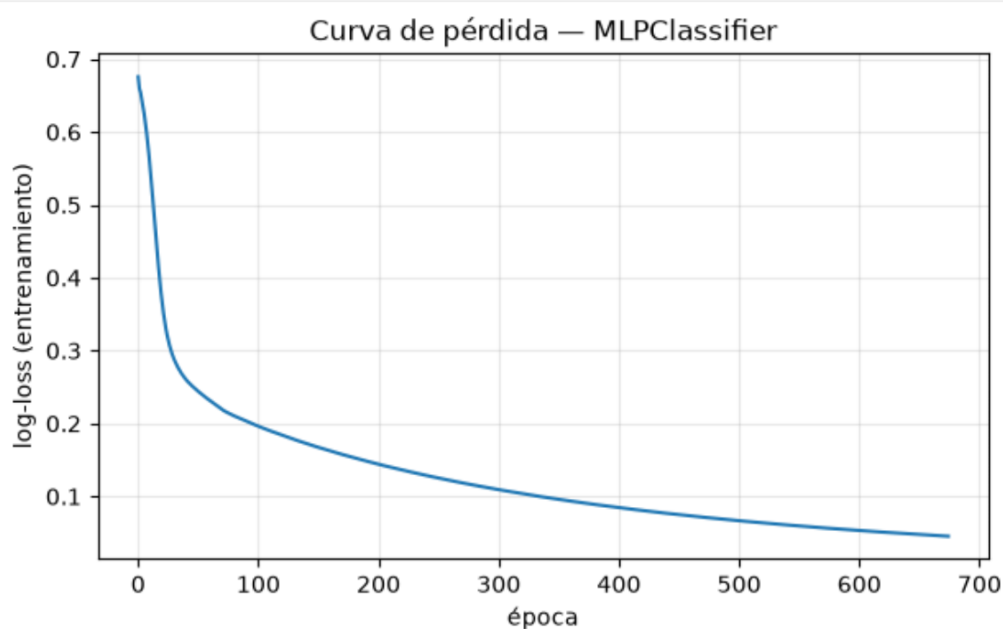
Luego, el log-loss mide qué tan mal o bien predijo la red, comparando la probabilidad con la etiqueta real. Qué tan seguro esté el modelo

Ejercicio 3 Instanciar, entrenar y evaluar

1. Se crea el modelo con los hiperparámetros de la tabla con `MLPClassifier`
2. Se entrena el modelo con `.fit(X_train, y_train)`
3. `model.score(X, y)`: → Usa el modelo ya entrenado para predecir la clase de cada imagen en `X`
→ Compara esas predicciones contra las etiquetas reales `y`.

`log-loss final = 0.0453` → Modelo confiado, pero con algunas dudas

log-loss	Probabilidad Promedio implícita	Qué significa
0	100%	Confianza perfecta
0.0453	~95.6%	Modelo confiado, pero con algunas dudas
0.69	50%	Modelo no confiado
> 0	≈ 0%	Modelo seguro pero equivocado



6. Clasificador como termómetro

Sabiendo que softmax devuelve dos probabilidades que suman 1, el método `.predict_proba(x)` es justo la forma en que scikit learn entrega las probabilidades directamente

→ `model.predict_proba(x)` : devuelve un arreglo

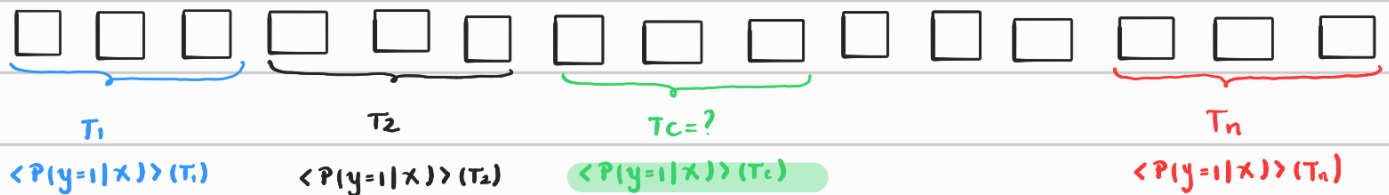
$$\underbrace{(n_imagenes, 2)}_{n \text{ imágenes}} \rightarrow \begin{bmatrix} P(y=0|x) & P(y=1|x) \\ \vdots & \vdots \end{bmatrix}$$

→ `model.predict_proba(x)[:, 1]` : Devuelve solo la columna de probabilidades de que estén desordenadas.

→ Se deben tomar todas las imágenes que comparten una misma T luego se calcula la probabilidad $P(y=1|x)$ y finalmente se promedian esos números. El resultado es un solo número por temperatura $\langle P(y=1|x) \rangle(T)$.

Esto indica: En promedio, qué tan seguro esté el modelo de que las configuraciones a esa T son desordenadas

→ T



Probabilidades dispersas
Se define la estimación de T_c como
la T que cruce por 0.5

≈ 0

≈ 0.5

≈ 1

7. Datos de barrido fino alrededor de T_c

temperatura_critica.npz : archivo comprimido de numpy que tiene
6200 configuraciones 16×16 en 31 temperaturas
entre 1.6 - 3.2 con 4 realizaciones

No tiene etiqueta de entrenamiento, solo las de
referencia

temperatura_critica.npz = { "X" : valores (6200 configuraciones)
"T" : valores
"realization" : valores
"y_ref" : valores }

$$X_{Tc} = \begin{bmatrix} \text{conf 1} \\ \text{conf 2} \\ \vdots \\ \text{conf 2600} \end{bmatrix}$$

(6200, 256)

$$T_{Tc} = \begin{bmatrix} T_1 \\ T_2 \\ \vdots \\ T_{6200} \end{bmatrix}$$

(6200, 1)

→ Algunas de estas T
se repiten. Hay solo
31 valores de T
diferentes.

realizacion_tc $\rightarrow$ (6200, 1)

y_ref_tc $\rightarrow$ (6200, 1)

$\rightarrow$ `idx1 = list(model.classes_).index(1)`

- `model.classes_` : atributo que `MLPClassifier` calcula automáticamente y guarda después de que se entrena. Contiene las clases distintas que el modelo vio en el `y_train`.
`array([0, 1])` Es un arreglo con 2 elementos

- `list(model.classes_)` : se convierte a lista

- `list(model.classes_).index(1)` : De esa lista, busca el número 1 y devuelve la posición del índice.
`[0, 1].index(1)  $\Rightarrow$  1.`

$\rightarrow$ `p1_tc = model.predict_proba(x_tc)[ :, idx1]`

pasa las 6200 configuraciones por el modelo ya entrenado, calcula las 2 probabilidades para cada una y se queda con columna que representa qué tan seguro esté el modelo de que cada imagen es fase desordenada.

$\rightarrow$ `predict_tc` : vector de 6200 números, 1 ó 0

Luego se compara posición por posición la decisión del modelo contra la de referencia. Da otro arreglo de True / False para las que coinciden. Luego se promedia el arreglo

De las 6200 configuraciones de barrido fino, la decisión final del modelo coincidió con la etiqueta de referencia en el 82.47% de los casos

Ejercicio 4 : Curva <P> vs T

Crear data frame con : T, realization, prob

Agrupar por realization y T.

→ .agg : calcular las estadísticas dentro de cada agrupación

Ejm :

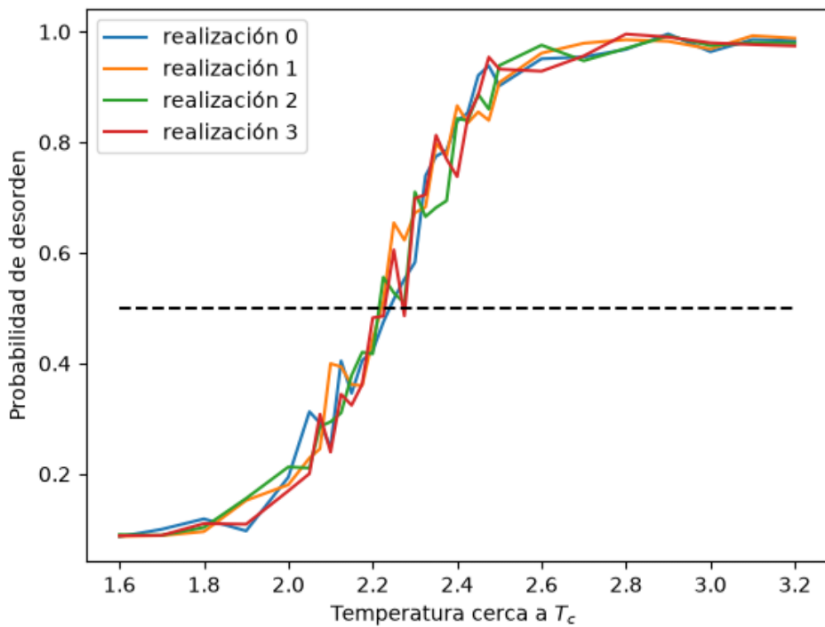
realización	T	prob
0	2.1	0.4
0	2.1	0.55
0	2.1	0.60
⋮		

	realización	T	prob	std	N
.agg →	0	2.1	0.52	0.089	6

	realizacion	T	prob_media	prob_std	N
0	0	1.6	0.086512	0.000394	50
1	0	1.7	0.099844	0.073550	50
2	0	1.8	0.118671	0.142166	50
3	0	1.9	0.096877	0.044426	50
4	0	2.0	0.194088	0.231677	50
...	...	...	...	...	...
119	3	2.8	0.995798	0.015673	50
120	3	2.9	0.990329	0.028237	50
121	3	3.0	0.979873	0.087987	50
122	3	3.1	0.976689	0.065616	50
123	3	3.2	0.974316	0.122123	50

Resumen

Al graficar por realización se obtiene



Las curvas de cada realización son consistentes.

El cálculo de T_c no depende de la realización usada

Ejercicio 5: Estimar T_c por interpolación

Para cada realización, encontrar el primer par de temperaturas consecutivas donde $\langle P \rangle$ cruza 0.5 e interpole linealmente

$$T_c \approx T_i + (0.5 - P_i) \frac{T_{i+1} - T_i}{P_{i+1} - P_i}$$

→ `def cruce_05` : recibe arreglo de T y probabilidades

$T = [1.6 \quad 2.3 \quad \dots \quad 3.2 \quad 1.6]$

$Prob = [\quad \quad \quad \quad \quad]$

- Recorre cada elemento del array y para en el penultimo elemento porque en cada vuelta compara el punto i con el $i+1$

- $\text{probs}[i] - 0.5 > 0$ si $\text{probs}[i] > 0.5$
 $\text{probs}[i] - 0.5 < 0$ si $\text{probs}[i] < 0.5$

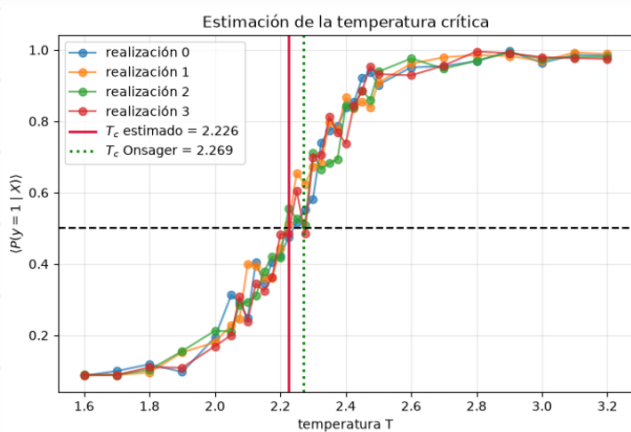
Al multiplicarlos :

$\begin{matrix} (+)(+) \\ (-)(-) \end{matrix} \} > 0$ ambos por encima de 0.5
 o por debajo.

$\begin{matrix} (+)(-) \\ (-)(+) \end{matrix} \} < 0$ uno por debajo y otro por encima.

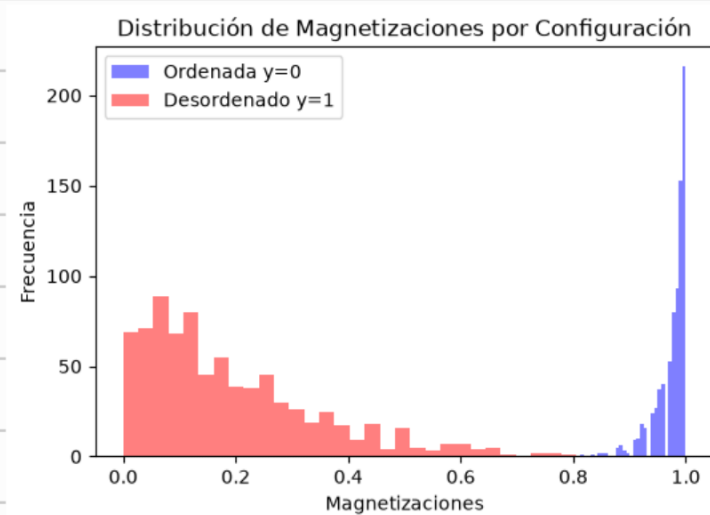
La curva cruza 0.5 entre esos 2 puntos

Aplicada esta función a la función de cruce por cada realización se obtiene la T_c . Luego se promedian para obtener T_c estimada



8. Análisis Crítico

1. Sesgo del estimador. El T_c estimado suele quedar por debajo de Onsager ≈ 2.23 vs 2.26 . Mencionar al menos 2 causas: tamaño finito $L=16$, definición de la etiqueta y, rango de temperaturas de entrenamiento, uso del umbral fijo 0.5



← T Mayor Temperatura Menor Temperatura →

1, 2, 3 en el notebook.

4. La diferencia entre `prob_media` y `pred.mean()`:

`prob_media` → promedio de las probabilidades crudas, es decir, las continuas entre 0 y 1.

`pred.mean()` → promedio de las predicciones (0 ó 1) después de aplicar el umbral de 0.5 a cada configuración individual.

